AMD AI DEVMASTER HACKATHON 2026

AMD Quant Assistant

A private, auditable quantitative investment Agent running locally on AMD Radeon GPU and ROCm.

24 / 24	98.1%	232	32K
CN evaluation	token accuracy	tests passed	served context

One-line pitch

The model proposes; deterministic code verifies; an independent Risk Agent has veto authority.

Track 2 - Development and Local Deployment of Private AI Agents

Submission repository: github.com/gxinxing/Radeon-hackathon-2026-07

1. Application scenario

Domestic-market research is full of natural-language ideas that cannot be trusted until they are grounded, structured, measured, and risk checked. The project turns an ordinary user request into a traceable decision workflow while keeping data and inference local.

- General questions receive a natural assistant response instead of being forced into a strategy template.
- Quantitative requests use RAG, structured DSL, deterministic validation, backtesting, and risk reporting.
- The reproducible demo uses deterministic synthetic domestic-market data and never places real orders.

Target users

User	Need	Delivered experience
Individual researcher	Turn an idea into a measurable strategy	Natural language to validated DSL and backtest
Quant team	Audit model behavior	Repair logs, sources, constraints, and risk verdict
Private deployment owner	Keep inference local	ROCm vLLM endpoint and self-hosted interfaces

2. Agent architecture

USER	→	INTENT ROUTER	→	RETRIEVAL AGENT	→	REASONING AGENT	→	RISK AGENT
		General answer		RAG + confidence		Qwen + LoRA		Independent veto

Execution path: natural language → RAG → strategy DSL → canonicalization → schema and semantic validation → deterministic backtest → independent risk report.

3. Core Agent capabilities

Capability	Implementation	Evidence
Reasoning	ReAct Thought / Action / Observation loop	src/agent/core.py
Planning	Intent routing and dynamic tool sequencing	src/agent/orchestrator.py
Tool use	Knowledge, indicators, validation, backtest, report	src/agent/tools.py
Memory	Working, episodic, and semantic tiers	src/agent/memory.py
Task execution	DSL, backtest, walk-forward, risk verdict	src/dsl and src/backtest
Self-healing	Allow-listed diagnose, repair, verify, rollback graph	scripts/graph_engine.py

4. Model and local deployment

Qwen2.5-7B is adapted with FP16 LoRA for domestic-market strategy generation. The merged model is served by vLLM through an OpenAI-compatible endpoint, allowing Open WebUI, Dify, and the Python Agent runtime to use the same private model asset.

Layer	Configuration
Base model	Qwen2.5-7B-Instruct
Adapter	LoRA r=64, alpha=128, dropout=0.05
Serving	vLLM FP16, model name models/qwen-trader-merged
Endpoint	http://127.0.0.1:8000/v1
Interfaces	Open WebUI, Dify six-node workflow, optional Gradio

5. AMD Radeon and ROCm optimization

Item	Measured configuration
GPU	AMD Radeon Graphics, gfx1100, 48 GiB VRAM
ROCm	7.2.1
Training	400 samples, 39 steps, 615 seconds
Peak training memory	16.21 GB
Inference	vLLM FP16, 32,768-token served context
Runtime	ROCm-native local inference; no hosted model API

Optimization choices

- FP16 LoRA limits training cost while preserving the full local base model.
- vLLM provides continuous batching and an OpenAI-compatible interface for multiple Agent consumers.
- Context compaction at 20K tokens and 512/128 RAG chunking prevent long-chat overflow.
- The Graph Engine verifies services and repairs only allow-listed configuration with rollback.

Measured evaluation

Metric	Before	Final
Overall pass rate	45.83%	100% (24/24)
JSON validity	75%	100%
Instrument match	70.83%	100%
Constraint compliance	45.83%	100%
Average latency	10.24 s	8.30 s

6. Safety, reproducibility, and limitations

- Risk constraints are implemented in deterministic code and cannot be overridden by model text.
- RAG content is bounded, source-aware, and confidence gated.
- No live order is submitted; paper mode is explicitly labeled simulation.
- Synthetic data makes the system path reproducible, but does not prove real investment performance.
- The 24/24 score includes transparent typed canonicalization and validation, not unconstrained raw-model accuracy.

Reproduction

1. Place the merged model at models/qwen-trader-merged. 2. Run bash scripts/setup.sh to start the API and vLLM. 3. Run bash scripts/verify_submission.sh. 4. Review artifacts/cn_market_eval_reproduced.json.

Deliverables

Deliverable	Location
Source and tests	src/, tests/, scripts/
Training and serving	training/
Dify workflow and tools	dify/
Evaluation evidence	artifacts/
Technical documentation	README.md and docs/

Disclaimer

This project is a hackathon research demonstration. It is not investment advice, does not connect to a live brokerage account, and does not promise trading returns.